CC3092 – Laboratorio 9: Series de tiempo y forecasting

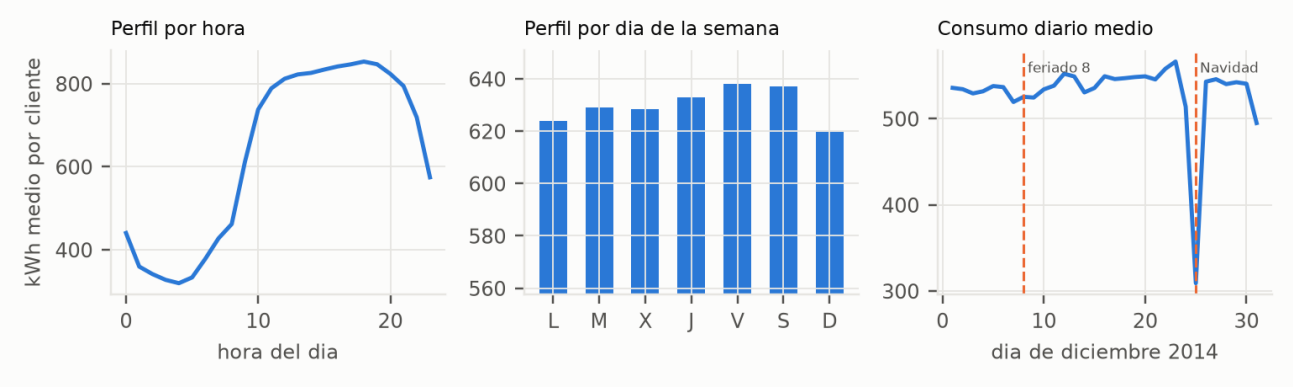
Consumo eléctrico por hora (ElectricityLoadDiagrams20112014), horizonte $H = 24$ h

Angel Esquit

Universidad del Valle de Guatemala – 1 de noviembre de 2026

1. Preparación de la serie y auditoría de fugas

Datos. Los 140,256 registros de 15 min (kW) se pasaron a kWh (×0,25) y se sumaron por hora. Con el filtro 2012–2014 y datos completos desde 2012-01-01 quedan **320 de 370 clientes**, **26,304 horas** por serie y **8,417,280** observaciones. Los modelos usan el **mismo subconjunto de 100 clientes** (semilla 42), por cómputo (GPU de 6 GB). Partición cronológica: entrenamiento hasta 2014-06-30, validación hasta 2014-09-30, prueba 2014-10-01 a 2014-12-31. **Cambio de horario.** En marzo la hora 01:00 no existe (queda con $\approx 4\,\%$ del consumo): se marca faltante y se interpola; en octubre la 01:00 acumula dos horas ($\approx 2\times$): se reemplaza por el promedio de la misma hora ± 7 días. **Escalas.** El consumo medio va de 2.4 a 48,817 kWh (mediana 147; razón $\approx 2 \times 10^4$): un z-score global haría que unos pocos clientes dominen la pérdida, por lo que se usa **z-score por serie con μ, σ solo de entrenamiento** (y, como alternativa, por ventana); las métricas se reportan siempre en kWh. **Calendario:** hora, día de la semana y mes con seno/coseno (2 dimensiones por variable en vez de 24+7+12 del one-hot, y preserva la ciclicidad) y una binaria de feriado de Portugal; son deterministas, conocidas a futuro. La figura muestra el perfil diario (valle nocturno, meseta 10–20 h), el semanal (domingo $\approx -3\,\%$ vs. viernes) y diciembre: el 25 el consumo cae a ≈310 kWh frente a ≈540 de un día normal ($-43\,\%$); 24–26 dic: 477 kWh y 31 dic–2 ene: 489 contra una media global de 636 ($\approx -25\,\%$). **Ventanas** ($L = 168$, $H = 24$, stride 1 h): 320 series → 6,943,040 / 699,200 / 699,200 (train/val/test); subconjunto de 100 clientes → **2,169,700 / 218,500 / 218,500**.



Auditoría de fugas (cada una verificada con `assert` en el notebook). (1) *Split aleatorio*: el corte es cronológico y ningún target de un split cae en el siguiente. (2) *Escalado con toda la serie*: μ, σ solo de entrenamiento. (3) *Early stopping con la prueba*: solo se usa validación; la prueba se evalúa una vez por modelo. (4) *Covariables mal alineadas*: la entrada es $[t - L, t - 1]$, el target $[t, t + H - 1]$ y solo el calendario determinista se usa a futuro.

2. Investigación de herramientas

PyTorch. `nn.LSTM/GRU`: `input_size`, `hidden_size`, `num_layers`, `batch_first` (entrada (B, L, F)), `dropout` (entre capas; solo si `num_layers`>1). Salidas: `output` $(B, L, D \cdot h)$, `h_n` y `c_n` $(D \cdot \text{capas}, B, h)$ (la GRU no tiene `c_n`). `nn.Conv1d` causal: se rellenan $(k-1)d$ ceros solo a la izquierda y `padding=0`; el campo receptivo de una TCN es $RF = 1 + \sum_i (k-1)d_i$ (con $k = 3$, $d_i = 2^i$, una conv por nivel: $1 + 2(2^n - 1)$; $n = 7$ cubre $L = 168$, $n = 6$ solo 127 h). `nn.TransformerEncoderLayer`: `d_model`, `nhead` (divide a `d_model`), `dim_feedforward`, `dropout`; la codificación posicional (seno/coseno o aprendida) se *suma* al embedding porque la atención no tiene noción de orden (PatchTST agrupa antes en parches). **Nixtla/Chronos.** NeuralForecast/StatsForecast: formato largo (`unique_id, ds, y`); `cross_validation`: `n_windows` (orígenes), `step_size` (horas entre orígenes) y `refit` (reajustar en cada ventana o entrenar una vez). `BaseChronosPipeline.predict_quantiles(context, prediction_length, quantile_levels)` → $(B, H, |q|)$, zero-shot. **Conceptos.** *Local vs global*: un modelo por serie (se adapta, pero con pocos datos y costo $\propto N$) o uno para todas (comparte patrones). *Teacher forcing*: el decoder recibe el valor real previo en entrenamiento (converge rápido) pero usa su predicción en inferencia (*exposure bias*). *RevIN*: normaliza cada ventana y revierte a la salida; ayuda ante cambios de nivel. *Pinball*: $\ell_\tau = \max\{\tau(y - \hat{q}), (\tau - 1)(y - \hat{q})\}$, minimizada por el cuantil τ. *MASE vs WAPE*: ambas sin escala; MASE divide por el MAE del naive estacional de cada serie y promedia (iguala a los clientes, < 1 supera al naive); WAPE= $\sum |e| / \sum |y|$ pondera por tamaño y es robusta a $y \approx 0$.

3. Protocolo y métricas

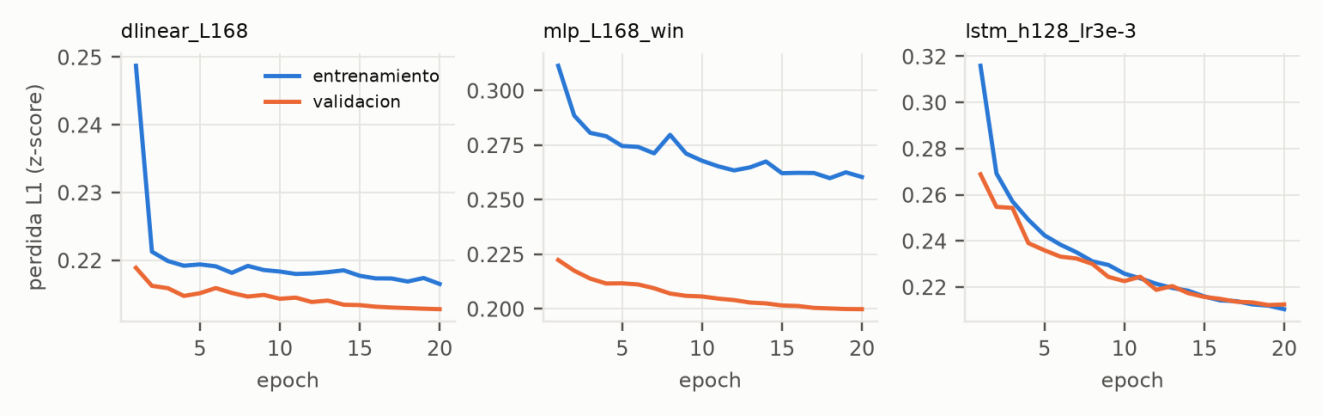
Una sola función `backtest(modelo)`: 92 orígenes diarios a las 00:00 (pesos entrenados una vez con datos hasta 2014-09-30). Métricas en kWh: MAE, RMSE, WAPE, MAPE y MASE (escala: MAE del naive de 24 h en entrenamiento, por cliente); MAE/RMSE por cliente, por paso h y por hora del día se guardan en `results/` (`per_client_metrics.csv`).

MAPE: solo $\approx 1\,\%$ de las horas de prueba son exactamente 0 (14 clientes) y se excluyen; aun así un error relativo individual llega a 785× y el MAPE se mueve de 9.3 % a 8.7 % al descartar $y < 10$ kWh, por lo que no se usa para comparar. **Reentrenamiento mensual** (1-oct, 1-nov, 1-dic, ventana expandible): MLP 0.8255→0.8268, iTransformer 0.8147→0.8037 y MSTL 0.9115→0.9133 de MASE: *el error casi no cambia* (cambios de +0,001, −0,011 y +0,002; el de iTransformer queda dentro de su desviación entre semillas, 0.016). Nota: MSTL ya se reajustaba en cada origen diario. El horizonte $H = 168$ no se evaluó (todos los modelos pronostican 24 h).

4. Iteraciones de hiperparámetros (solo validación)

Iteración	L	meses	h/cap.	lr	ép.	MAE_v	MASE_v	params	s/ép.	MB
dlinear L48	48	30	-	0.001	11	35.4	0.905	2352	1.3	63
dlinear L168	168	30	-	0.001	20	31.8	0.835	8112	1.3	113
dlinear L336	336	30	-	0.001	7	31.7	0.835	16176	1.3	181
dlinear lr3e-3	336	30	-	0.003	20	31.0	0.821	16176	1.3	181
dlinear m3	168	3	-	0.001	20	31.7	0.826	8112	1.3	113
nlinear L336	336	30	-	0.001	20	30.8	0.817	8088	1.0	181
mlp L168	168	30	256/2	0.001	20	30.8	0.816	158232	1.5	116
mlp L168 win	168	30	256/2	0.001	20	30.3	0.800	158232	1.5	118
lstm L168	168	30	64/1	0.001	20	37.6	0.989	24536	3.5	3825
lstm h128	168	30	128/1	0.001	20	35.2	0.925	77784	12.5	7534
lstm 2layers	168	30	64/2	0.001	20	36.7	0.970	57816	7.1	3846
lstm win	168	30	64/1	0.001	20	36.1	0.957	24536	3.9	3846
lstm m3	168	3	64/1	0.001	8	49.0	1.282	24536	3.7	3825
lstm h128 lr3e-3	168	30	128/1	0.003	20	32.7	0.855	77784	13.2	1920
lstm auto h128 lr3e-3	168	30	128/1	0.003	20	33.6	0.858	70785	15.5	1932
seq2seq h128 lr3e-3	168	30	128/1	0.003	12	35.8	0.928	141441	24.2	1921
itransformer L168	168	30	-	1e-3	1000 pasos	30.9	0.805	289944	71 (total)	273
nhits L168	168	30	-	1e-3	1000 pasos	32.2	0.833	815379	383 (total)	4695
tcn L168	168	30	-	1e-3	1000 pasos	33.9	0.890	74521	94 (total)	4755
patchtst L168	168	30	-	1e-3	1000 pasos	34.8	0.888	466843	115 (total)	4970
patchtst m3	168	3	-	1e-3	1000 pasos	34.2	0.867	466843	82 (total)	780
patchtst m12	168	12	-	1e-3	1000 pasos	34.4	0.881	466843	95 (total)	2181
itransformer 3k	168	30	-	1e-3	3000 pasos	31.5	0.816	289944	110 (total)	273
nhits 3k	168	30	-	1e-3	3000 pasos	31.5	0.806	815379	2079 (total)	4696
tcn 3k	168	30	-	1e-3	3000 pasos	31.8	0.825	74521	335 (total)	4755
patchtst 3k	168	30	-	1e-3	3000 pasos	33.6	0.863	466843	340 (total)	4970
deepar base	168	30	64/2	0.001	20	35.4	0.933	52419	10.9	1015
deepar lr3e-3	168	30	64/2	0.003	20	34.0	0.885	52419	11.0	1015
deepar h128 lr3e-3	168	30	128/2	0.003	20	32.9	0.844	203139	35.0	1948

Tabla 1: Iteraciones seleccionadas (completas en `results/iterations.csv`); MAE_v/MASE_v en validación; ép.=epochs; MB=memoria pico de GPU. Las filas de NeuralForecast (sin covariables) entrenan por pasos y su "s/ép." es el tiempo total; las últimas son DeepAR propio.



Pérdida y MASE. Menor pérdida de entrenamiento *no* implica mejor MASE: la LSTM h=128 (lr 3e-3) termina con pérdida de entrenamiento 0.210, menor que DLinear (0.217) y que el MLP con normalización por ventana (0.260), pero su MASE_v es peor (0.855 vs. 0.835 y 0.800; el MLP regulariza con dropout y las pérdidas bajo normalizaciones distintas no son estrictamente comparables). **Hallazgo NeuralForecast:** al pasar el calendario como `futr_exog_list`, NHITS y TCN divergieron (MASE_v 1.66 y 29.5, sin cambiar con lr 3e-3/3e-4 ni recorte de gradiente); sin exógenas convergen (0.833 y 0.890), por lo que los modelos de NeuralForecast se reportan sin covariables. Con 3000 pasos (filas "3k") NHITS mejora a 0.806, TCN a 0.825 y PatchTST a 0.863; iTransformer se detiene antes por early stopping y queda peor (0.816 vs. 0.805 con 1000 pasos). Para cada familia se usó la corrida de mejor MASE_v: NHITS 3k, TCN 3k, PatchTST 3k e iTransformer 1k.

5.1 Estrategias multi-paso (LSTM $h = 128$, lr 3e-3). MAE de $h = 1$–$6 \rightarrow h = 19$–24: autorregresiva 22.8→40.2 (+76 %), multi-salida 24.6→37.0 (+50 %), encoder–decoder 25.8→42.1 (+63 %). La autorregresiva gana en los primeros pasos ($h = 1$: 21.2 vs. 26.3) y pierde desde $h \approx 10$: sí se observa acumulación de error. El MAE tiene además un pico a $h \approx 17$–20 (tarde-noche) por el ciclo diario, así que no crece de forma monótona.

5. Resultados en prueba

Modelo	L	Estrategia	Cov.	Params	t_{ent} (s)	t_{inf} (s)	HW	MAE	RMSE	WAPE	MASE	MASE_{22-28}
Chronos-Bolt	512	directa	no	47 718 016	0	6.7	GPU	28.1	89.3	0.059	0.760	1.36
NHITS	168	multi-salida	no	815 379	2079	6.2	GPU	29.8	92.1	0.062	0.804	1.37
iTransformer	168	multi-salida	no	289 944	71	5.4	GPU	30.0	91.0	0.063	0.815	1.41
MLP	168	multi-salida	si	158 232	32	0.1	GPU	30.5	92.7	0.064	0.826	1.47
DeepAR	168	autorregresiva	si	203 139	935	177.4	GPU	31.2	96.5	0.065	0.836	1.35
TCN	168	multi-salida	no	74 521	335	7.5	GPU	31.0	97.8	0.065	0.837	1.52
NLinear	336	multi-salida	no	8 088	21	0.1	GPU	32.0	99.9	0.067	0.853	1.80
DLinear	336	multi-salida	no	16 176	27	0.1	GPU	32.1	99.9	0.067	0.855	1.80
LSTM (AR)	168	autorregresiva	si	70 785	425	1.2	GPU	32.7	101.9	0.068	0.857	1.46
LSTM (MS)	168	multi-salida	si	77 784	289	0.3	GPU	31.8	96.8	0.066	0.858	1.38
PatchTST	168	multi-salida	no	466 843	340	8.5	GPU	32.0	98.1	0.067	0.864	1.60
LightGBM	168	recursiva	si	1 497	99	5.3	CPU	32.5	105.6	0.068	0.867	1.59
MSTL	1344	directa	no	0	0	603.2	CPU	35.2	109.2	0.073	0.912	1.92
Seq2seq LSTM	168	encoder-decoder	si	141 441	360	1.2	GPU	35.2	104.0	0.073	0.917	1.38
SNaive24	24	directa	no	0	0	0.0	CPU	39.1	129.3	0.082	1.043	2.20
SNaive168	168	directa	no	0	0	0.0	CPU	39.6	124.6	0.083	1.081	1.60
Naive	1	directa	no	0	0	0.0	CPU	152.3	376.0	0.318	4.091	4.08

Tabla 2: Prueba (92 orígenes, 100 clientes), ordenada por MASE. MS=multi-salida, AR=autorregresivo; Cov.=usa calendario; t_{ent} y t_{inf} (92 orígenes) en segundos; HW = hardware (GPU GTX 1660 SUPER o CPU). Chronos: solo inferencia, 47.7 M parámetros. Los modelos de NeuralForecast se evaluaron con pesos entrenados con datos hasta 2014-09-30. $\text{MASE}_{22\text{-}28}$: semana del 22 al 28 de dic.

Probabilístico. Chronos-Bolt: pinball 9.37 kWh y cobertura P10–P90 de 80.4 % (nominal 80 %, bien calibrado). **DeepAR** (implementación propia en PyTorch: LSTM de 2 capas, cabeza Student-t, calendario como covariables, 100 trayectorias muestreadas): MASE 0.836, pinball 10.55 kWh y cobertura 74.5 % (algo sobreconfiado). `NeuralForecast.DeepAR` no fue viable en la GPU de 6 GB (60 GB solicitados en validación y $>$ 25 min sin completar 300 pasos). No se evaluó TFT (se usó iTransformer).

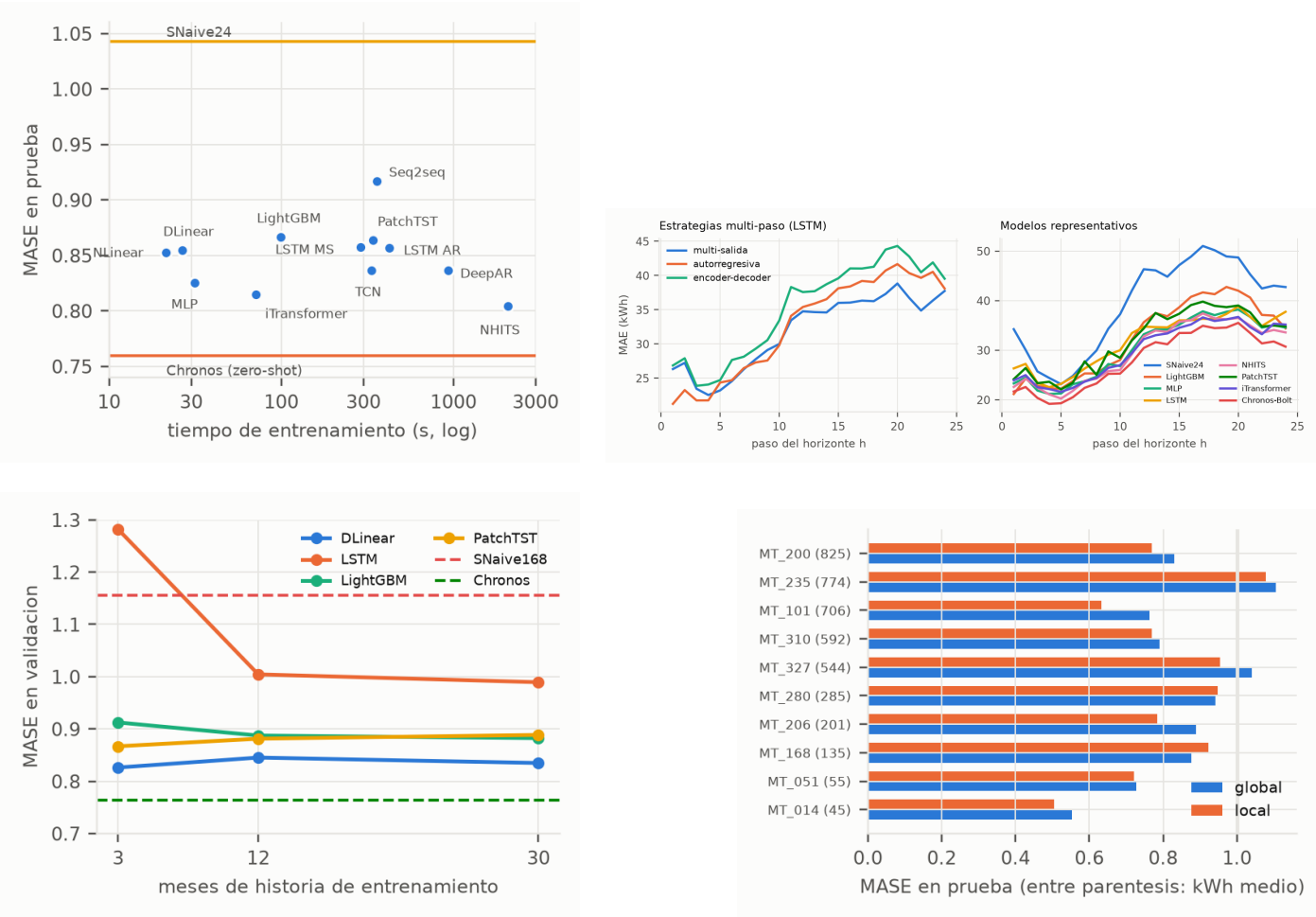


Figura 1: Arriba: MASE de prueba vs. tiempo de entrenamiento (escala log; líneas: Chronos y SNaive24) y MAE por paso del horizonte. Abajo: MASE de validación vs. meses de historia (líneas punteadas: SNaive168 y Chronos) y MASE local vs. global por cliente (kWh medio entre paréntesis).

5.3 Local vs global (MLP, 10 clientes, semilla fija): MASE global **0.852** vs. local **0.808** (MAE 23.5 vs. 21.9 kWh). El local gana en 8 de 10; el global solo en MT_168 y MT_280 (clientes pequeños, 135 y 285 kWh medios) y por poco. Los 5 clientes $\geq$ 500 kWh mejoran con el modelo local (p. ej. MT_101: 0.763→0.632). Costo: 10 modelos locales 177 s vs. 32

s del global. **Semillas** (3): MLP 0.823 ± 0.002; iTransformer 0.812 ± 0.016 (0.806, 0.831, 0.800, reajustando dentro de `cross_validation`; la tabla muestra 0.815 con el checkpoint de 1000 pasos).

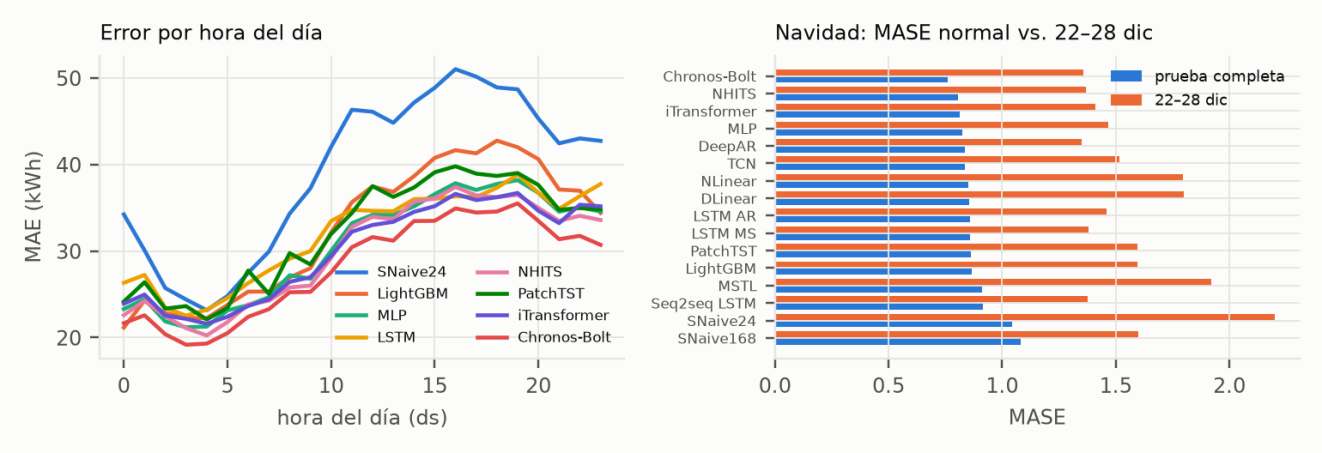


Figura 2: MAE por hora del día de ocho modelos representativos (el pico de error está en la tarde-noche) y MASE de toda la prueba contra el de la semana 22–28 de diciembre para los 16 modelos (sin Naive).

6. Discusión y conclusiones

Ganador. Chronos-Bolt zero-shot (MASE 0.760; MAE 28.1) superó al seasonal naive 24 h (1.043; MAE 39.1) en un 27 %, sin entrenar; el mejor modelo entrenado (NHITS, 0.804, pero 2079 s de entrenamiento) lo supera en 23 %, iTransformer (0.815, 71 s) en 22 % y el MLP (0.826, 32 s, 158 k parámetros) en 21 %. Sí se justifica pasar del naive a un modelo: el MLP cuesta segundos y el reentrenamiento mensual no aportó, por lo que el mantenimiento es bajo; NHITS gana solo 0.011 de MASE sobre iTransformer a 30 veces el costo. En la frontera de Pareto quedan Chronos, NLinear, MLP, iTransformer y NHITS; DeepAR, TCN y las LSTM quedan dominadas.

Ruido. Con ±0,016 de desviación entre semillas en iTransformer, las diferencias entre los 6 mejores entrenados (0.804–0.853) no son concluyentes; solo Chronos y la brecha con los baselines lo son.

DLinear. Se replica parcialmente: DLinear (0.855) supera por poco a PatchTST (0.864), a LightGBM (0.867) y a las LSTM; PatchTST es mejor en h=1–6 (23.8 vs. 24.6) y DLinear en h=19–24 (36.3 vs. 36.6). No supera a NHITS, iTransformer ni Chronos (34.8, 35.2 y 32.9 en h=19–24).

Estrategia multi-paso. La multi-salida fue la mejor y más estable en h largos; la autorregresiva solo gana a $h \leq 9$. El error crece del tramo h=1–6 al 19–24 un 47 % (DLinear), 54–60 % (NeuralForecast, Chronos) y 74–76 % en los dos modelos autorregresivos (LSTM AR y DeepAR): la acumulación de error es mayor en esa estrategia.

Ventana, normalización e historia (validación). L: DLinear 0.905 (L=48) vs. 0.835 (168/336); MLP 0.816→0.803 y NLinear 0.835→0.817 con L=336; la LSTM no cambia. Normalización por ventana: MLP 0.816→0.800, LSTM 0.989→0.957, DLinear sin efecto. Historia (3/12/30 meses): DLinear 0.826/0.845/0.835 (3 meses basta), LightGBM 0.912/0.887/0.882, LSTM 1.282/1.004/0.989; el SNaive168 vale 1.156 y Chronos 0.763. DLinear supera a LightGBM desde 3 meses; la LSTM supera a los estadísticos desde 12 meses y nunca a LightGBM. PatchTST (con 1000 pasos): 0.867/0.881/0.888, también mejor con menos historia; de 12 a 30 meses ningún modelo mejora más de 0.015, así que la ganancia de más historia es pequeña salvo en la LSTM entre 3 y 12 meses.

Local vs global. No convino el global en nuestros datos (0.852 vs. 0.808): con un MLP de 158 k parámetros y 100 clientes heterogéneos, la capacidad compartida parece limitar a los clientes grandes; solo dos pequeños se benefician del global. Es un experimento de 10 clientes y una semilla.

Chronos. Quedó *por delante* de todos los modelos entrenados (0.760 vs. 0.804) y con mejor calibración (80.4 % vs. 74.5 % de DeepAR). Lo usaríamos al arrancar un cliente nuevo, sin infraestructura de entrenamiento o como baseline; entrenar tendría sentido para usar calendario/covariables, modelar cuantiles propios o ajustar a un cliente.

Navidad/Año Nuevo. El MASE de 22–28 dic sube de ≈0.8 a 1.36–2.2 en *todos* los modelos (SNaive24 2.20, DLinear/NLinear 1.80, MLP 1.47, LSTM MS 1.38, DeepAR 1.35, Chronos 1.36): hubo un cambio de régimen general. Los modelos con calendario (MLP, LSTM, LightGBM, DeepAR) no superan sistemáticamente a los que no lo usan (Chronos, iTransformer, NHITS), y el feriado marca solo el día 25 mientras la caída dura varios días.

Limitaciones. 100 clientes; 1000–3000 pasos en los modelos de NeuralForecast, que van sin calendario (comparación no homogénea con los propios); DeepAR es implementación propia, no la de NeuralForecast; no se evaluó TFT ni $H = 168$; una sola partición de prueba.

Repositorio: https://github.com/AngelEsquit/Lab9-DL